

Практическая работа «REST API для управления задачами (TaskFlow API)»

Описание работы

С нуля создать REST API для сервиса управления задачами (аналог Trello/Jira в миниатюре). Проект состоит из трёх уровней сложности: каждый следующий включает предыдущий и добавляет новые требования.

Стек: ASP.NET Core, EF Core (SQLite), Swagger, .http-файлы.

Модель данных (общая для всех уровней)

Project: Id, Name, Description, CreatedAt

TaskItem: Id, ProjectId, Title, Description, Status, AssignedToId, DueDate, CreatedAt

AppUser: Id, Username, Email

Comment: Id, TaskItemId, AuthorId, Content, CreatedAt

Связи:

- Project → TaskItem (1:M)
 - AppUser → TaskItem (1:M, через AssignedToId)
 - TaskItem → Comment (1:M)
 - AppUser → Comment (1:M, через AuthorId)
-

Уровень 1: Лёгкий — «Базовый CRUD и REST»

Цель: научиться строить простые REST-эндпоинты, возвращать правильные статус-коды и подключить Swagger.

Задание 1.1. Создать проект и настроить Swagger

Создать ASP.NET Core Web API проект TaskFlowApi. Подключить Swagger с XML-комментариями.

Требования:

- В .csproj включить генерацию XML-документации.
- В Program.cs подключить AddSwaggerGen с чтением XML-комментариями.
- Swagger UI должен открываться по адресу /swagger.
- Создать три Entity-класса: Project, TaskItem, AppUser (без связей, просто поля).

Проверка:

- Открыть /swagger — увидеть пустой список эндпоинтов.
 - Нет ошибок компиляции.
-

Задание 1.2. CRUD для проектов

Создать контроллер ProjectsController с полным CRUD.

Эндпоинты:

Метод	URL	Что делает	Код ответа
GET	/api/projects	Список всех проектов	200
GET	/api/projects/{id}	Один проект по ID	200 или 404
POST	/api/projects	Создать проект	201 + Location
PUT	/api/projects/{id}	Обновить проект целиком	204 или 404
DELETE	/api/projects/{id}	Удалить проект	204 или 404

Требования:

- Использовать [ApiController] и [Route("api/[controller]")].
- Возвращать ActionResult<T>.
- При POST возвращать 201 Created с заголовком Location.
- При GET несуществующего ID возвращать 404 NotFound.
- Написать XML-комментарии (/// <summary>) над каждым методом.
- Добавить [ProducesResponseType] для каждого возможного кода ответа.
- Хранить проекты в памяти (статический список List<Project>).

Проверка через .http-файл:

```
@baseUrl = https://localhost:7001

### Создать проект
POST {{baseUrl}}/api/projects
Content-Type: application/json

{ "name": "Веб-сайт", "description": "Корпоративный портал" }

### Получить все проекты
GET {{baseUrl}}/api/projects

### Получить проект по ID
GET {{baseUrl}}/api/projects/1

### Обновить проект
PUT {{baseUrl}}/api/projects/1
Content-Type: application/json

{ "name": "Веб-сайт v2", "description": "Обновлённый портал" }

### Удалить проект
DELETE {{baseUrl}}/api/projects/1
```

Задание 1.3. CRUD для задач

Создать контроллер TasksController с тем же набором эндпоинтов, но для TaskItem.

Дополнительно:

- Задача имеет поле Status (enum: ToDo, InProgress, Done).
- При POST если Status не указан — по умолчанию ToDo.
- При PUT если Status не указан — возвращай 400 Bad Request (обязательное поле при полной замене).

Проверка через .http-файл:

```
### Создать задачу
POST {{baseUrl}}/api/tasks
Content-Type: application/json

{ "title": "Сверстать главную", "status": "ToDo", "projectId": 1 }

### Получить задачу
GET {{baseUrl}}/api/tasks/1
```

Задание 1.4. Валидация и ProblemDetails

Добавить валидацию через Data Annotations:

- Name проекта — обязательно, минимум 3 символа, максимум 100.
- Title задачи — обязательно, минимум 5 символов.
- DueDate задачи — если указана, должна быть в будущем.

Требования:

- Использовать атрибуты [Required], [StringLength], [Range] и т.д.
- Благодаря [ApiController] валидация работает автоматически и вернёт 400 с ValidationProblemDetails.
- В Swagger должны быть видны описания полей из XML-комментарий.

Проверка:

```
### Создать проект с пустым именем → 400
POST {{baseUrl}}/api/projects
Content-Type: application/json

{ "name": "", "description": "test" }
```

Ожидаемый ответ:

```
{
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Name": ["The Name field is required."]
  }
}
```

Чек-лист уровня 1

- Проект запускается, Swagger открывается.
 - CRUD для проектов и задач работает (проверено через .http).
 - Возвращаются правильные статус-коды (200, 201, 204, 404, 400).
 - Валидация работает: пустое имя → 400 с ProblemDetails.
 - XML-комментарии видны в Swagger.
 - [ProducesResponseType] указан для всех эндпоинтов.
-

Уровень 2: Средний — «DTO, EF Core, связи, версионирование»

Цель: перейти от хранений в памяти к базе данных, разделить Entity и DTO, настроить связи и версионирование.

Задание 2.1. Подключить EF Core с SQLite

Создать ApplicationDbContext с DbSet<Project>, DbSet<TaskItem>, DbSet<AppUser>.

Требования:

- Установи NuGet-пакеты: Microsoft.EntityFrameworkCore.Sqlite, Microsoft.EntityFrameworkCore.Design.
- Настрой DbContext в Program.cs через AddDbContext<AppDbContext>.
- Создай миграцию и обнови базу (dotnet ef migrations add Init, dotnet ef database update).
- В OnModelCreating настрой:
 - Связь Project → TaskItem (1:M) с внешним ключом.
 - Каскадное удаление: при удалении проекта удаляются его задачи.
 - Индекс по TaskItem.Status (для быстрого фильтра).
 - Seed data: один проект и одну задачу при первом запуске.

Проверка:

- База taskflow.db создаётся.
 - При GET /api/projects возвращается seed-проект.
 - При DELETE проекта — его задачи тоже удаляются (проверить через БД).
-

Задание 2.2. Разделить Entity и DTO

Создай отдельные DTO для каждого эндпоинта.

Требуемые DTO:

DTO	Назначение	Поля
ProjectDto	Ответ (GET, POST)	Id, Name, Description, CreatedAt, TaskCount
CreateProjectDto	Создание (POST)	Name, Description
UpdateProjectDto	Обновление (PUT)	Name, Description
TaskItemDto	Ответ	Id, Title, Description, Status, ProjectId, AssignedToId, DueDate
CreateTaskDto	Создание	Title, Description, Status, ProjectId, AssignedToId, DueDate
UpdateTaskDto	Обновление	Title, Description, Status, AssignedToId, DueDate

Правила:

- Entity AppUser содержит Email и PasswordHash. В TaskItemDto отдавай только AssignedToId и Username (через вложенный DTO или плоско — на выбор, но PasswordHash не отдаём).
- Маппинг — ручной (extension-методы). AutoMapper — по желанию.
- ProjectDto.TaskCount — количество задач в проекте (вычисляется, а не хранится).

Проверка:

```
### Создать проект через DTO
POST {{baseUrl}}/api/projects
Content-Type: application/json
```

```
{ "name": "Мобильное приложение", "description": "iOS + Android" }
```

```
### Проверить, что в ответе нет лишних полей
GET {{baseUrl}}/api/projects/1
```

В ответе НЕ должно быть навигационных свойств EF Core (например, Project.Tasks как массив сущностей). Только TaskCount как число.

Задание 2.3. Комментарии и вложенные ресурсы

Добавь сущность Comment и контроллер для работы с комментариями задачи.

Эндпоинты (вложенный ресурс):

Метод	URL	Что делает
GET	/api/tasks/{taskId}/comments	Список комментариев задачи
POST	/api/tasks/{taskId}/comments	Добавить комментарий
DELETE	/api/comments/{id}	Удалить комментарий

Требования:

- Если задача не найдена → 404 с ProblemDetails.
- При GET возвращай список CommentDto с AuthorUsername (не AuthorId + Email).
- При POST тело — CreateCommentDto с полем Content (обязательно, 1–500 символов).
- Настрой связь TaskItem → Comment (1:M) с каскадным удалением.

Проверка:

```
### Добавить комментарий к задаче 1
POST {{baseUrl}}/api/tasks/1/comments
Content-Type: application/json

{ "content": "Нужно уточнить ТЗ", "authorId": 1 }

### Получить комментарии задачи 1
GET {{baseUrl}}/api/tasks/1/comments

### Комментарии к несуществующей задаче → 404
GET {{baseUrl}}/api/tasks/999/comments
```

Задание 2.4. Версионирование API

Добавь версионирование через URL-сегмент.

Требования:

- Установи пакет Asp.Versioning.Mvc.
- v1 — текущий контракт (без изменений).
- v2 — для задач добавь поле Priority (enum: Low, Medium, High) и убери Description из ответа.
- Создай два контроллера: TasksController (v1) и TasksV2Controller (v2) — или один с двумя методами через [MapToApiVersion].
- В Swagger должны отображаться обе версии.
- Пометь v1 как Deprecated = true — в Swagger будет предупреждение.

Проверка:

```
### v1 — есть Description, нет Priority
GET {{baseUrl}}/api/v1/tasks/1

### v2 — есть Priority, нет Description
GET {{baseUrl}}/api/v2/tasks/1
```

Чек-лист уровня 2

- EF Core подключён, SQLite работает, миграции применены.
 - Seed data создаётся при первом запуске.
 - Entity и DTO разделены, PasswordHash не попадает в ответы.
 - Комментарии работают как вложенный ресурс.
 - Каскадное удаление работает (проект → задачи → комментарии).
 - Версионирование настроено, v1 deprecated, v2 добавляет Priority.
 - Swagger показывает обе версии.
-

Уровень 3: Сложный — «Идемпотентность, пагинация, фильтрация, глобальная обработка»

Цель: довести API до продакшен-уровня: устойчивость к повторам, поиск с пагинацией, централизованная обработка ошибок.

Задание 3.1. Пагинация, фильтрация и сортировка

Добавь расширенный GET для задач с параметрами.

Эндпоинт:

GET

/api/v2/tasks?page=1&pageSize=10&status=InProgress&sortBy=dueDate&sortDir=desc&search=верст

Параметры:

Параметр	Тип	Описание
page	int	Номер страницы (по умолчанию 1)
pageSize	int	Размер страницы (по умолчанию 10, максимум 50)
status	string	Фильтр по статусу (ToDo, InProgress, Done)
priority	string	Фильтр по приоритету (Low, Medium, High)
sortBy	string	Поле для сортировки (title, dueDate, createdAt)
sortDir	string	Направление (asc, desc)
search	string	Поиск по Title (содержит)

Требования:

- Верни PagedResult<TaskItemDto>:

```
public class PagedResult<T>
{
    public List<T> Items { get; set; }
    public int Page { get; set; }
    public int PageSize { get; set; }
    public int TotalCount { get; set; }
    public int TotalPages => (int)Math.Ceiling(TotalCount / (double)PageSize);
}
```

- Используй Skip и Take в EF Core запросе.
- Сортировка — через switch по sortBy (не Dynamic LINQ, а явный switch).
- Если pageSize > 50 — возвращай 400 Bad Request.
- Напиши XML-комментарии для всех параметров — они появятся в Swagger.

Проверка:

```
### Первая страница, статус InProgress, сортировка по дате
GET
{{baseUrl}}/api/v2/tasks?page=1&pageSize=5&status=InProgress&sortBy=dueDate&sortDir=desc
```

```
### Поиск по слову
GET {{baseUrl}}/api/v2/tasks?page=1&pageSize=10&search=верст
```

```
### pageSize больше 50 → 400
GET {{baseUrl}}/api/v2/tasks?page=1&pageSize=100
```

Задание 3.2. Идемпотентность для создания задач

Защити POST /api/v2/tasks от дубликатов через Idempotency-Key.

Требования:

- Заголовок X-Idempotency-Key (GUID) — обязателен при POST.
- Если заголовок отсутствует → 400 Bad Request с ProblemDetails.
- Сервер хранит пары (key, response) в БД (таблица IdempotencyRecords: Key, RequestBodyHash, ResponseBody, StatusCode, CreatedAt).
- Если ключ уже есть в БД:
 - Если тело запроса совпадает → верни сохранённый ответ (тот же статус и тело).
 - Если тело отличается → 409 Conflict с ProblemDetails («Idempotency key reused with different body»).
- Если ключа нет → обработай запрос, сохрани ответ в БД, верни его.
- Реализуй через middleware или action filter.

Проверка:

```
### Первый запрос — создаёт задачу
POST {{baseUrl}}/api/v2/tasks
X-Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
Content-Type: application/json

{ "title": "Новая задача", "priority": "High", "projectId": 1 }

### Повтор с тем же ключом и телом → тот же ответ (201, та же задача)
POST {{baseUrl}}/api/v2/tasks
X-Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
Content-Type: application/json

{ "title": "Новая задача", "priority": "High", "projectId": 1 }

### Тот же ключ, но другое тело → 409 Conflict
POST {{baseUrl}}/api/v2/tasks
X-Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
Content-Type: application/json

{ "title": "Другая задача", "priority": "Low", "projectId": 1 }

### Без ключа → 400
POST {{baseUrl}}/api/v2/tasks
Content-Type: application/json

{ "title": "Без ключа", "priority": "Medium", "projectId": 1 }
```

Задание 3.3. Глобальная обработка ошибок

Реализуй единый обработчик исключений через IExceptionHandler.

Требования:

- Создай класс GlobalExceptionHandler : IExceptionHandler.
- Зарегистрируй его в Program.cs:

```
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
builder.Services.AddProblemDetails();
app.UseExceptionHandler();
```

- Логируй исключение через ILogger.

- Возвращай 500 с ProblemDetails, но БЕЗ stack trace и БЕЗ деталей исключения в поле detail (только «Внутренняя ошибка сервера»).
- Создай собственное исключение NotFoundException (для случаев «задача не найдена» и т. п.) и обрабатывай его отдельно — возвращай 404 с ProblemDetails.
- Создай собственное исключение BusinessException (например, «нельзя завершить задачу без комментариев») — возвращай 422 Unprocessable Entity с ProblemDetails.

Проверка:

```
### Запрос к несуществующей задаче → 404 ProblemDetails
GET {{baseUrl}}/api/v2/tasks/9999
```

```
### Попытка завершить задачу без комментариев → 422
PATCH {{baseUrl}}/api/v2/tasks/1/status
Content-Type: application/json
```

```
{ "status": "Done" }
```

Если у задачи нет комментариев, сервер должен вернуть:

```
{
  "title": "Business rule violation",
  "status": 422,
  "detail": "Нельзя завершить задачу без комментариев"
}
```

Задание 3.4. CORS для внешнего клиента

Настрой CORS, чтобы фронтенд на <http://localhost:3000> мог обращаться к API.

Требования:

- В Program.cs добавь политику CORS с разрешённым origin <http://localhost:3000>.
- Разреши любые методы и заголовки.
- app.UseCors() должен стоять ДО UseAuthorization, но ПОСЛЕ UseRouting.

Проверка:

- Из браузера на localhost:3000 сделай `fetch('http://localhost:7001/api/v2/tasks')`.
 - Запрос не должен блокироваться CORS-ошибкой.
-

Чек-лист уровня 3

- Пагинация, фильтрация и сортировка работают.
 - `pageSize > 50` возвращает 400.
 - Idempotency-Key защищает от дублей: тот же ключ + то же тело = тот же ответ.
 - Тот же ключ + другое тело = 409 Conflict.
 - Отсутствие ключа = 400.
 - Глобальный обработчик ловит исключения и возвращает ProblemDetails без stack trace.
 - NotFoundException → 404, BusinessException → 422.
 - CORS настроен для localhost:3000.
-

Структура проекта

TaskFlowApi/

